

LEADER NUMBER:

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the size of the array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter the array elements:");
        for (int i = 0; i < n; i++)
        {
            arr[i] = sc.nextInt();
        }
        int max = arr[n - 1];
        ArrayList<Integer> leaders = new ArrayList<>();
        leaders.add(max);
        for (int i = n - 2; i >= 0; i--)
        {
            if (arr[i] >= max)
            {
                max = arr[i];
                leaders.add(arr[i]);
            }
        }
        Collections.reverse(leaders);
        System.out.println("Leader elements are:");
        for (int leader : leaders) {
            System.out.print(leader + " ");
        }
    }
}
```

OUTPUT:

```
Enter the size of the array: 6
Enter the array elements:
16 17 4 3 5 2
Leader elements are:
17 5 2
```

REVERSE THE LAST K ELEMENTS:

```
import java.util.*;
public class Main
```

```

{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        ArrayList<Integer> list = new ArrayList<>();
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++)
        {
            list.add(sc.nextInt());
        }
        System.out.print("Enter k: ");
        int k = sc.nextInt();
        if (k <= 0 || k > n)
        {
            System.out.println("Invalid value of k");
            return;
        }
        ArrayDeque<Integer> deque = new ArrayDeque<>();
        Stack<Integer> stack = new Stack<>();
        for (int i = n - k; i < n; i++)
        {
            deque.addLast(list.get(i));
        }
        while (!deque.isEmpty())
        {
            stack.push(deque.removeFirst());
        }
        for (int i = n - k; i < n; i++)
        {
            list.set(i, stack.pop());
        }
        System.out.println("Array after reversing last " + k + " elements:");
        for (int num : list)
        {
            System.out.print(num + " ");
        }
    }
}

```

OUTPUT:

Enter size of array: 5

Enter array elements:

10 20 30 40 50

Enter k: 3

Array after reversing last 3 elements:

10 20 50 40 30

REMOVE THE DUPLICATES:

```
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String str = sc.nextLine();
        Set<Character> set = new LinkedHashSet<>();
        for (char ch : str.toCharArray())
        {
            set.add(ch);
        }
        System.out.print("String after removing duplicates: ");
        for (char ch : set) {
            System.out.print(ch);
        }
    }
}
```

OUTPUT:

Enter a string: programming

String after removing duplicates: progamin

UNION:

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of first array: ");
        int n1 = sc.nextInt();
        Set<Integer> set = new LinkedHashSet<>();
        System.out.println("Enter first array elements:");
        for (int i = 0; i < n1; i++)
        {
            set.add(sc.nextInt());
        }
        System.out.print("Enter size of second array: ");
        int n2 = sc.nextInt();
    }
}
```

```

        System.out.println("Enter second array elements:");
        for (int i = 0; i < n2; i++)
        {
            set.add(sc.nextInt());
        }
        System.out.println("Union: " + set);
    }
}

```

OUTPUT:

```

Enter size of first array: 5
Enter first array elements:
1 2 3 4 5
Enter size of second array: 6
Enter second array elements:
0 9 8 7 5 6
Union: [1, 2, 3, 4, 5, 0, 9, 8, 7, 6]

```

INTERSECTION:

```

import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of first array: ");
        int n1 = sc.nextInt();
        Set<Integer> set1 = new LinkedHashSet<>();
        System.out.println("Enter first array elements:");
        for (int i = 0; i < n1; i++)
        {
            set1.add(sc.nextInt());
        }
        System.out.print("Enter size of second array: ");
        int n2 = sc.nextInt();
        Set<Integer> intersection = new LinkedHashSet<>();
        System.out.println("Enter second array elements:");
        for (int i = 0; i < n2; i++)
        {
            int num = sc.nextInt();
            if (set1.contains(num))
            {
                intersection.add(num);
            }
        }
    }
}

```

```

    }
    System.out.println("Intersection: " + intersection);
}
}

```

OUTPUT:

```

Enter size of first array: 5
Enter first array elements:
1 2 3 4 5
Enter size of second array: 6
Enter second array elements:
1 2 3 4 5 6
Intersection: [1, 2, 3, 4, 5]

```

CONTAINS DUPLICATE II

```

import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++)
        {
            arr[i] = sc.nextInt();
        }
        System.out.print("Enter window size (k): ");
        int k = sc.nextInt();
        HashSet<Integer> set = new HashSet<>();
        boolean found = false;
        for (int i = 0; i < n; i++)
        {
            if (set.contains(arr[i]))
            {
                found = true;
                break;
            }
            set.add(arr[i]);
            if (set.size() > k)
            {
                set.remove(arr[i - k]);
            }
        }
    }
}

```

```
        }  
    }  
    System.out.println(found);  
}  
}
```

OUTPUT:

Enter size of array: 6

Enter array elements:

1 2 4 5 6 5

Enter window size (k): 3

true